极客时间

AI 编程范式转换

规范驱动开发（SDD）+ Harness Engineering
无状态推理 + 有状态编排

第 1 周 · 第 1 节

讲师：黄佳

极客时间黄佳老师出品

极客时间

课程愿景

不是教你用工具
是教你像架构师一样设计 AI 系统

工具会过时，工程思想才是你真正的资产

这门课的两条线

极客时间

思想线 · PPT 讲义

讲架构、讲设计、讲「为什么」

底层框架 + 设计决策

适合想理解底层逻辑的同学

帮你在工具迭代时快速迁移

行动线 · 动手实操

Step by step 带着做

截图 + 步骤 + 预期结果

适合先动手再理解的同学

照着做就能跑通

极客时间

极客时间

心态提示

跑通固然可喜
理解架构才是核心收获

代码两年就淘汰 · 设计思想用一辈子 · 卡住了先跳过，回头再来

四周·三次跃迁

阶段	周次	你在做什么	身份转变
用 Agent 编程	Week 1	写规范，让 AI 按规范生成代码	使用者
自动化运营	Week 2	加 Hooks/Pipeline/CI-CD	运营者
设计 Agent 系统	Week 3-4	多 Agent 协作 + 平台部署	设计者

课程代码仓库：https://github.com/huangjia2019/ai-knowledge-base

底层架构：规范 → Harness → 模型

极客时间

SDD 三层规范（你写的）

AGENTS.md	agents/*.md	skills/*/SKILL.md
项目规范	角色规范	能力规范

↓ 加载

Harness （运行时容器）

OpenCode / Claude Code / Cursor

循环 · 记忆 · 行动 · 权限控制

↓ 调用

模型（推理引擎）

DeepSeek / Qwen / Claude / GPT

无状态 · 纯推理 · f(input) → output

Software 3.0 · 范式转换

极客时间

Software 1.0 — 手写每一行代码（if–else, for loops）

Software 2.0 — 用数据训练模型替代手写规则（ML / Deep Learning）

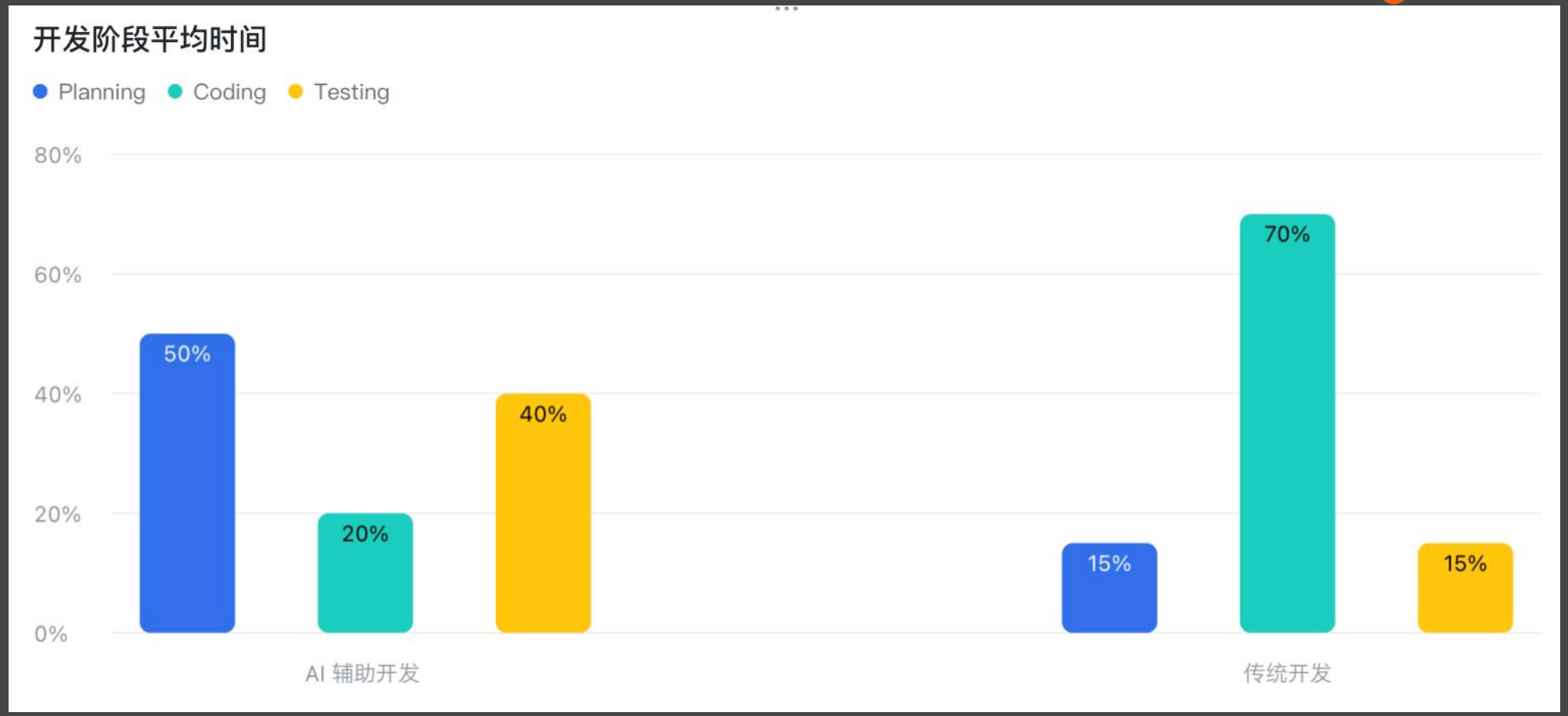
Software 3.0 — 用自然语言描述意图，AI 生成实现（LLM + Agent）

范式转换的本质：

你不再是“代码的生产者”，而是“意图的传达者”和“质量的把关者”

新比例：Planning 40% → Coding 20% → Testing 40%

斯坦福 CS146S · AI 辅助软件开发的时间分配



核心发现：编码时间压缩到 20%，但规划和测试时间翻倍
启示：你需要更强的“架构设计能力”和“验证能力”

极客时间

设计反转（Design Inversion）

传统：人写代码，AI 辅助
新范式：人写规范，AI 写代码

80% 时间想清楚要什么（写规范），20% 时间验证 AI 做得对不对

实现力高原（Implementation Plateau）

极客时间

AI 的写代码能力已经进入高原期

- GPT-4 → GPT-4o → Claude 4，SWE-bench 提升幅度：个位数百分比
- 不是不进步了，而是”够用了”

瓶颈不在模型，在你能不能把需求说清楚

- 十个人用同样的模型、同样的 API 额度，产出差异可以有十倍
- 区别不在模型，在规范

极客时间

极客时间

核心观点

编程 ≠ 写代码
编程 = 设计意图 × 精确表达 × 验证产出

Design → Prompt → Validate → Iterate

AI Coder 全景图 · 2025~2026

极客时间

工具	类型	开源	国产模型	Memory	Agent 编排
OpenCode	终端 Agent	MIT 开源	原生支持	AGENTS.md	原生支持
Claude Code	终端 Agent	闭源	不支持	CLAUDE.md	原生支持
Cursor	IDE 插件	闭源	部分支持	.cursorrules	Composer
Windsurf	IDE 插件	闭源	部分支持	Rules	Cascade
Copilot	IDE 插件	闭源	不支持	.github/copilot	有限
Trae	IDE 插件	闭源	原生支持	Rules	Builder

极客时间黄佳老师出品

工程化能力 6 维对比

极客时间

能力维度	OpenCode	Claude Code	Cursor	说明
Memory 记忆	AGENTS.md	CLAUDE.md	.cursorrules	持久化项目上下文
Sub-Agents 委派	原生支持	原生支持	Composer	任务分解与委派
Skills 技能	原生支持	原生支持	自定义命令	可复用能力封装
Hooks 事件	原生支持	原生支持	有限	事件驱动自动化
MCP 协议	原生支持	原生支持	部分支持	外部工具集成
Headless CI/CD	原生支持	原生支持	不支持	无人值守运行

为什么选 OpenCode 为主线?

极客时间

MIT 开源 — 代码完全公开，可学习架构设计，可二次开发

100K+ Stars — 社区活跃，生态成熟，问题有人答

国产模型原生支持 — DeepSeek / Qwen / GLM / Kimi 零成本接入

概念映射完整 — 与 Claude Code 的六大能力一一对应

零成本方案 — 开源工具 + 国产模型 API，学习门槛降到最低

核心原则：学概念、学思想，不绑定任何一个工具

极客时间

OpenCode vs Claude Code · 概念映射表

极客时间

概念	OpenCode	Claude Code	本质
项目记忆	AGENTS.md	CLAUDE.md	持久化上下文注入
子 Agent	Sub-Agent 原语	SubAgent 原语	隔离上下文的任务委派
技能包	Skill 文件	SKILL.md	声明式能力封装
快捷命令	Slash Command	Slash Command	用户触发的预定义流程
事件钩子	Hook 配置	Hook 配置	工具执行前后的自动化
外部工具	MCP Server	MCP Server	标准化外部能力接入
无人值守	Headless 模式	Headless 模式	CI/CD 集成
编程 SDK	API / SDK	Agent SDK	程序化调用 Agent

极客时间

工程思想

无状态推理 + 有状态编排

极客时间

极客时间

模型的本质：无状态函数

极客时间

大语言模型 = f(input) → output

- 每次调用都是独立的，没有记忆，没有状态

三个关键约束：

- **无记忆** — 上一轮对话说了什么，模型完全不知道（除非你重新喂给它）
- **无决策** — 模型不会主动决定「下一步做什么」，它只回答当前问题
- **无行动** — 模型不能读文件、不能调 API、不能执行任何操作

本质：模型是一个极其强大但完全被动的推理引擎

编排器的本质：有状态系统（Harness）

极客时间

编排器 = while True: 观察 → 思考 → 行动 → 更新状态

Harness（马具） — 把模型的"智力"转化为"行动力"的中间层

- 马有力气，但没有马具，拉不动车。Harness 改变的是力量的传导方式

Harness = 编排器 + 规范化 （Tools + Knowledge + Observation + Action Interfaces + Permissions）

三个核心能力（模型没有的）：

- **循环** — 持续运行，直到任务完成或用户中断
- **记忆** — 维护对话历史、项目上下文、工具调用结果
- **行动** — 读写文件、执行命令、调用 API、与外部系统交互

OpenCode / Claude Code / Cursor 的本质都是编排器，都实现了Harness

- 它们负责管理状态、调度模型、执行工具、维护上下文，并提供权限系统和确保运行时安全

极客时间黄佳老师出品

被低估的关键能力：上下文管理

极客时间

模型上下文窗口有限（200K tokens），真实任务很快撑满

Harness 的解决方案：自动压缩 + 重注入

- 对话历史接近上限 → 早期消息压缩成摘要
- 但 AGENTS.md / CLAUDE.md 内容 → 每次压缩后重新注入

这就是为什么你写的规范"一直有效"

- 不是模型记住了它，是 Harness 在每次压缩后都重新塞给模型

下一节讲 Memory 工程时，你会深入理解这个机制

极客时间

极客时间黄佳老师出品

极客时间

一个重要命题

Harness 比模型更重要

极客时间

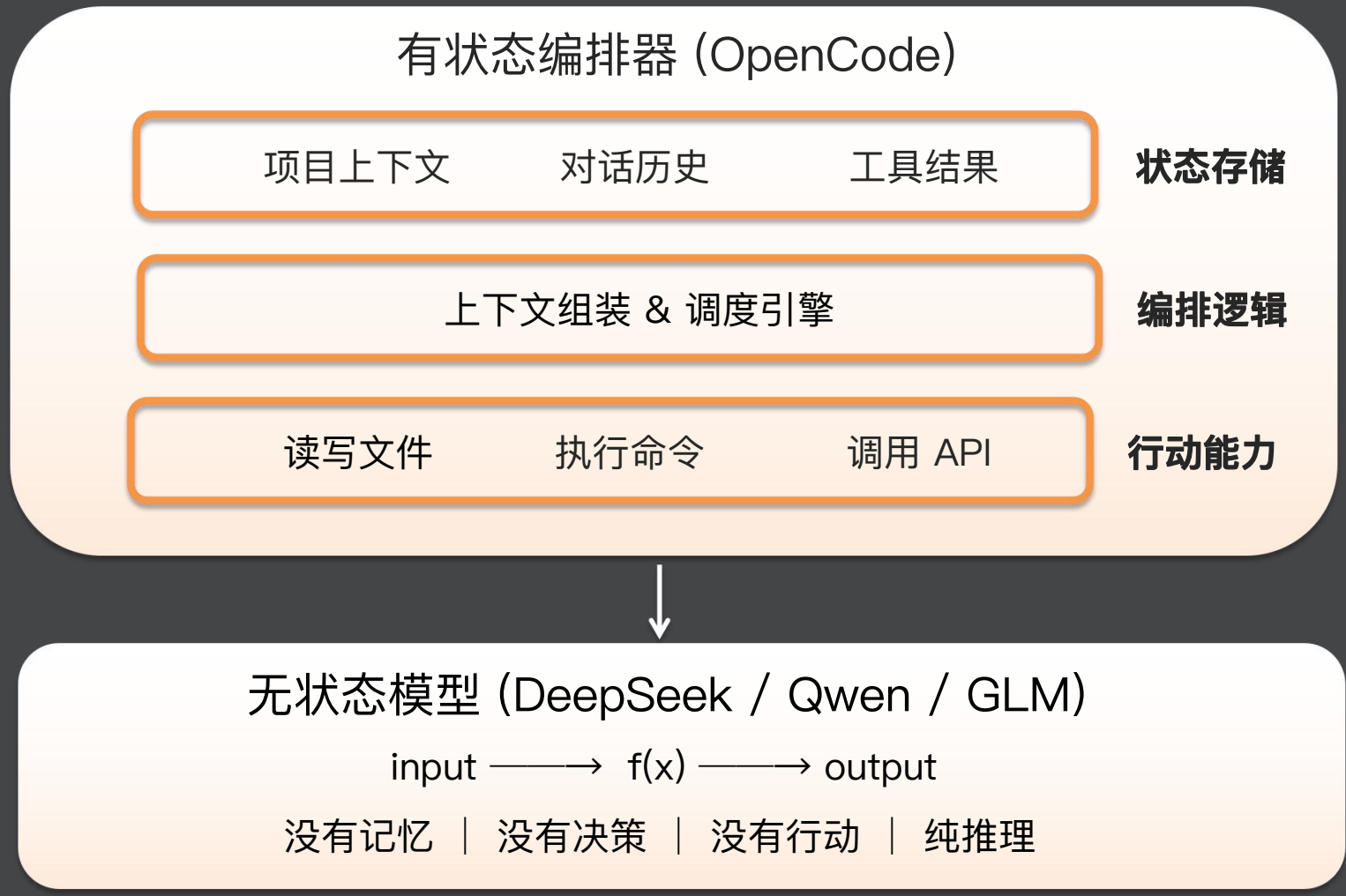
极客时间

同一个模型在不同 Harness 中的表现差距
远大于不同模型在同一个 Harness 中的差距

你调教 Harness 的能力 = 你真正的杠杆点

极客时间黄佳老师出品

核心架构：无状态模型 + 有状态编排器



极客时间黄佳老师出品

实战验证：裸 API 调用 vs OpenCode 编排

极客时间

裸 API 调用（无状态）

直接调用 DeepSeek API
每次请求独立，无上下文
不知道项目背景
不能读写文件
不能执行命令
一问一答，做完就忘

= 只有推理，没有系统

OpenCode 编排（有状态）

通过 OpenCode 调用同一个模型
自动注入 AGENTS.md 上下文
了解项目结构和规范
可以读写项目文件
可以执行终端命令
多轮交互，持续迭代

= 推理 + 记忆 + 行动 = 系统

极客时间黄佳老师出品

行动营项目 — AI 知识库助手系统

极客时间

一个人 + AI Agent 团队 = 一个完整的知识管理系统

四大核心能力：

- **定制化知识采集** — GitHub / HN / 技术博客 / RSS 自动采集
- **AI 深度分析解读** — 摘要、技术亮点、架构分析、趋势研判
- **结构化知识存储** — 标签体系、全文检索、知识图谱
- **多渠道智能分发** — 公众号 / 飞书 / Telegram / 邮件

技术栈： OpenCode 编排 + 国产模型推理 + MCP 工具集成

极客时间

V1 → V4 渐进演进路线图

极客时间

Week 1 V1 编码自动化	Week 2 V2 采集自动化	Week 3 **V3 质量自动化**	Week 4 **V4 服务自动化**
AI 编写代码	自动化流水线	多 Agent 协作	全平台上线
Memory 配置	Hooks 触发	Agent 编排	MCP 集成
人驱动对话 AI 写码	Skills 标准化	协作协议定义	CI/CD 部署
第一批结构化知识	自动采集+分析	自动审核+分发	监控+运维
---工程思想--- 无状态 vs 有状态	---工程思想--- 事件驱动架构	---工程思想--- 分布式协作	---工程思想--- **系统可靠性**

极客时间黄佳老师出品

第 1 周课表总览

极客时间

节次	主题	核心内容	工程思想
第 1 节	AI 编程范式转换	工具选型 + 概念映射	无状态推理 + 有状态编排
第 2 节	Memory 工程	AGENTS.md 设计与分层	上下文工程原理
第 3 节	Sub-Agent 委派	任务分解与角色定义	关注点分离原则
第 4 节	Skill 封装	可复用能力包设计	声明式 vs 命令式

本节作业

极客时间

任务 1：安装 OpenCode 并完成环境配置

安装 OpenCode（参考 GitHub README），配置国产模型 API （推荐 DeepSeek，也可选 Qwen/GLM/Kimi）。截图验证：opencode --version + 第一次对话成功。

任务 2：对比实验：裸 API 调用 vs OpenCode 编排

用 Python 直接调用模型 API 完成一个小任务（如分析一段代码），再用 OpenCode 完成同一个任务。对比两者的差异，写一段 200 字的体会：你感受到了什么是“无状态”和“有状态”？

任务 3：阅读 OpenCode 源码中的编排循环（选学）

找到 OpenCode 源码中主循环（Agent Loop），理解它是如何实现“观察 → 思考 → 行动 → 更新状态”的。在群里分享你找到的关键代码片段。

本节小结

编程不再是写代码，而是写规范 + 验证结果

编程 = 设计意图 × 精确表达 × 验证产出

模型已经够强，差距在你能否把需求说清楚

80%想清楚要什么，20%让 AI 去实现

Agent = 推理 + 记忆 + 行动 + 各种工程护栏

Harness 比模型更重要，是你真正的杠杆

工具会过时，但工程思想不会

下一节：Memory 工程 — 让 AI 读懂你的项目

极客时间黄佳老师出品